

Final Semester Project.

Project Name

Patient Management System

Head Of The Group

Muhammad Ali.

Members Of The Group: (2)

Muhammad Ali.

Sharjeel Sohail.

List of “Cout”

1. Load_from_file function.

- `cout << "\n No existing data found. Starting fresh.\n";`
- `cout << "\n Loaded " << patient_count << " patient records from file.\n";`

2. Save_to_file function.

- `cout << "\n Error saving data to file.\n";`
- `cout << "\n Patient data saved to file.\n";`

3. Display_slots function.

- `cout << "\n Available Slots for Patients: " << remaining_slots << endl;`

4. Add_patient function.

- `cout << "\n -Enter Patient ID: ";`
- `cout << " -Enter Patient Name: ";`
- `cout << " -Enter Patient Age: ";`
- `cout << " -Enter Patient Gender (Male / Female): ";`
- `cout << " -Enter Patient Ailment: ";`
- `cout << "\n Patient added successfully!\n";`

5. View_patients function

- `cout << "\n No patients to display.\n";`

- `cout << "\n List of Patients:\n";` (once per function call)
- Inside the loop:
 - `cout << "\n |Patient ID: " << patient_ids[i];`
 - `cout << "\n |Name: " << patient_names[i];`
 - `cout << "\n |Age: " << patient_ages[i];`
 - `cout << "\n |Gender: " << patient_genders[i];`
 - `cout << "\n |Ailment: " << patient_ailments[i];`
 - `cout << "\n -----";`

6. **Search_patient** function.

- `cout << "\n No patients to search.\n";`
- `cout << "\n Enter Patient \"ID\" to search: ";`
- Inside the loop:
 - `cout << "\n Patient Found:\n";`
 - `cout << " Patient ID: " << patient_ids[i] << endl;`
 - `cout << " Name: " << patient_names[i] << endl;`
 - `cout << " Age: " << patient_ages[i] << endl;`
 - `cout << " Gender: " << patient_genders[i] << endl;`
 - `cout << " Ailment: " << patient_ailments[i];`
- After the loop (if not found):
 - `cout << "\n Patient \"ID\" " << id << " not found.\n";`

7. **Delete_patient** function.

- `cout << "\n No patients to delete.\n";`
- `cout << "\n Enter Patient \"ID\" to delete: ";`
- Inside the loop:

- `cout << "\n Patient \"ID\" \" << id << \" deleted successfully.\n\";`
- After the loop (if not found):
 - `cout << "\n Patient \"ID\" \" << id << \" not found.\n\";`

8. **Menu** function

- `cout << "\n Patient Management System.\n\";`
- `cout << \" 1. Add Patient.\n\";`
- `cout << \" 2. View All Patients.\n\";`
- `cout << \" 3. Search Patient \"ID\".\n\";`
- `cout << \" 4. Delete Patient \"ID\".\n\";`
- `cout << \" 5. Display Remaining Slots.\n\";`
- `cout << \" 6. Exit.\n\";`
- `cout << \" Enter your Choice: \";`
- `cout << "\n Exiting... \n\";`
- `cout << "\n Invalid choice. Please try again.\n\";`

Functions

- `void load_from_file()`
- `void save_to_file()`
- `void display_slots()`
- `void add_patient()`
- `void view_patients()`
- `void search_patient()`
- `void delete_patient()`
- `void menu()`
- `int main()`

Variables

Global Variables:

- `int max_patients = 100;`
- `int patient_ids[100];`
- `string patient_names[100];`
- `int patient_ages[100];`
- `string patient_genders[100];`
- `string patient_ailments[100];`
- `int patient_count = 0;`

Local Variables:

1. `load_from_file.`
 - `int id;`
2. `save_to_file.`
 - `int i;` (used in the for loop)
3. `display_slots`
 - `int remaining_slots;`
4. `add_patient.`
 - `int age;`
5. `view_patients`
 - `int i;` (used in the for loop)
6. `search_patient`
 - `int i;` (used in the for loop)
 - `int id;` (to store user input)
7. `delete_patient.`
 - `int i;` (used in the outer for loop)
 - `int j;` (used in the inner for loop)

- **int id;** (to store user input)
8. **menu**
- **int choice;**
9. **main**
- **No additional int variables.**

Code For Patient Management System

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

// Global Variables
int max_patients = 100;
int patient_count = 0;

int patient_ids[100];
string patient_names[100];
int patient_ages[100];
string patient_genders[100];
string patient_ailments[100];

// Function to load patient data from file
void load_from_file() {
```

```
ifstream infile("patients.txt");

if (!infile.is_open()) {
    cout << "\n No existing data found. Starting fresh.\n";
    return;
}

patient_count = 0; // Reset patient count
while (infile >> patient_ids[patient_count]) {
    infile.ignore(); // Ignore leftover newline.
    getline(infile, patient_names[patient_count]);
    infile >> patient_ages[patient_count];
    infile.ignore();
    getline(infile, patient_genders[patient_count]);
    getline(infile, patient_ailments[patient_count]);
    patient_count++;
}
infile.close();
cout << "\n Loaded " << patient_count << " patient records from file.\n";
}

// Function to save patient data to file
void save_to_file() {
    ofstream outfile("patients.txt");
    if (!outfile.is_open()) {
        cout << "\n Error saving data to file.\n";
        return;
    }
}
```

```
}
```

```
for (int i = 0; i < patient_count; i++) {  
    outfile << patient_ids[i] << endl;  
    outfile << patient_names[i] << endl;  
    outfile << patient_ages[i] << endl;  
    outfile << patient_genders[i] << endl;  
    outfile << patient_ailments[i] << endl;
```

```
}
```

```
outfile.close();
```

```
cout << "\n Patient data saved to file.\n";
```

```
}
```

```
// Function to display remaining slots
```

```
void display_slots() {
```

```
    int remaining_slots = max_patients - patient_count;
```

```
    cout << "\n Available Slots for Patients: " << remaining_slots << endl;
```

```
}
```

```
// Function to add a new patient
```

```
void add_patient() {
```

```
    if (patient_count >= max_patients) {
```

```
        cout << "\n No more space to add patients.\n";
```

```
        return;
```

```
}
```



```
cout << "\n -Enter Patient ID: ";  
cin >> patient_ids[patient_count];  
cin.ignore();
```

```
cout << " -Enter Patient Name: ";  
getline(cin, patient_names[patient_count]);
```

```
cout << " -Enter Patient Age: ";  
cin >> patient_ages[patient_count];  
cin.ignore();
```

```
cout << " -Enter Patient Gender (Male / Female): ";  
getline(cin, patient_genders[patient_count]);
```

```
cout << " -Enter Patient Ailment: ";  
getline(cin, patient_ailments[patient_count]);
```

```
patient_count++;  
cout << "\n Patient added successfully!\n";
```

```
save_to_file(); // Save data to file.  
display_slots();
```

```
}
```

```
// Function to view all patients
```

```
void view_patients() {  
    if (patient_count == 0) {
```

```
    cout << "\n No patients to display.\n";  
    return;  
}
```

```
cout << "\n List of Patients:\n";  
for (int i = 0; i < patient_count; i++) {  
    cout << "\n |Patient ID: " << patient_ids[i];  
    cout << "\n |Name: " << patient_names[i];  
    cout << "\n |Age: " << patient_ages[i];  
    cout << "\n |Gender: " << patient_genders[i];  
    cout << "\n |Ailment: " << patient_ailments[i];  
    cout << "\n -----";  
}  
}
```

// Function to search for a patient by ID

```
void search_patient() {  
    if (patient_count == 0) {  
        cout << "\n No patients to search.\n";  
        return;  
    }  
}
```

```
int id;  
cout << "\n Enter Patient \"ID\" to search: ";  
cin >> id;
```

```
for (int i = 0; i < patient_count; i++) {
```

```
    if (patient_ids[i] == id) {  
        cout << "\n Patient Found:\n";  
        cout << " Patient ID: " << patient_ids[i] << endl;  
        cout << " Name: " << patient_names[i] << endl;  
        cout << " Age: " << patient_ages[i] << endl;  
        cout << " Gender: " << patient_genders[i] << endl;  
        cout << " Ailment: " << patient_ailments[i] << endl;  
        return;  
    }  
}  
cout << "\n Patient \"ID\" " << id << " not found.\n";  
}
```

// Function to delete a patient by ID

```
void delete_patient() {  
    if (patient_count == 0) {  
        cout << "\n No patients to delete.\n";  
        return;  
    }  
  
    int id;  
    cout << "\n Enter Patient \"ID\" to delete: ";  
    cin >> id;
```

```
    for (int i = 0; i < patient_count; i++) {  
        if (patient_ids[i] == id) {  
            for (int j = i; j < patient_count - 1; j++) {
```

```
    patient_ids[j] = patient_ids[j + 1];
    patient_names[j] = patient_names[j + 1];
    patient_ages[j] = patient_ages[j + 1];
    patient_genders[j] = patient_genders[j + 1];
    patient_ailments[j] = patient_ailments[j + 1];
}
patient_count--;
cout << "\n Patient \"ID\" " << id << " deleted successfully.\n";

save_to_file(); // Save data to file.
display_slots();
return;
}
}
cout << "\n Patient \"ID\" " << id << " not found.\n";
}

// Function to display the menu
void menu() {
    load_from_file(); // Load data from file at the start.

    while (true) {
        cout << "\n Patient Management System.\n";
        cout << " 1. Add Patient.\n";
        cout << " 2. View All Patients.\n";
        cout << " 3. Search Patient \"ID\".\n";
        cout << " 4. Delete Patient \"ID\".\n";
```

```
cout << " 5. Display Remaining Slots.\n";
```

```
cout << " 6. Exit.\n";
```

```
cout << " Enter your Choice: ";
```

```
int choice;
```

```
cin >> choice;
```

```
switch (choice) {
```

```
case 1:
```

```
    add_patient();
```

```
    break;
```

```
case 2:
```

```
    view_patients();
```

```
    break;
```

```
case 3:
```

```
    search_patient();
```

```
    break;
```

```
case 4:
```

```
    delete_patient();
```

```
    break;
```

```
case 5:
```

```
    display_slots();
```

```
    break;
```

```
case 6:
```

```
    cout << "\n Exiting... \n";
```

```
    save_to_file(); // Save data before exiting.
```

```
    return;
```

```
        default:
            cout << "\n Invalid choice. Please try again.\n";
        }
    }
}

// Main function
int main() {
    menu();
    return 0;
}
```

Definitions

1. Global Variable

A variable declared outside all functions and accessible from any function in the program.

2. infile.ignore

Skips characters in the input file stream, either a specific number or until a certain character (like a newline) is found.

3. cin.ignore

Skips characters in the input from the user (console), often used to clear leftover characters like a newline.

4. outfile.is_open

Checks if the output file is successfully opened for writing.

5. outfile.close

Close the output file and ensure all data is saved properly.

6. infile.is_open

Checks if the input file is successfully opened for reading.

7. infile.close

Closes the input file and frees resources.

8. ifstream

A class in C++ is used for reading data from files.

9. ofstream

A class in C++ used for writing data to files.

10. infile

A variable name typically used to represent an object of the ifstream class for reading files.

11. outfile

A variable name typically used to represent an object of the ofstream class for writing files.